Software Design Description

EswatiniHomes Property Listing System

Group Members: Simelane Njabulo -202301338
Mathonsi Simiso-202302294
Malaza Mncobi-202201551
Vilakati Noncedo-202402458

Standard Alignment: IEEE Std 1016-2009

Contents

1 Introduction **2**
- 1.1 Purpose . . . 2
- 1.2 Scope . . . 2
- 1.3 Design Stakeholders and Concerns . . . 2

2 Design Viewpoints **2**
- 2.1 Context Viewpoint . . . 2
- 2.2 Composition Viewpoint . . . 3
- 2.3 Logical Viewpoint (Data Design) . . . 4
  - 2.3.1 Table: properties . . . 4
- 2.4 Interface Viewpoint . . . 5
  - 2.4.1 Public Search Interface (`index.php`) . . . 5
  - 2.4.2 Admin Delete Interface (`delete.php`) . . . 5
- 2.5 Interaction Viewpoint . . . 5
  - 2.5.1 Sequence: Property Deletion . . . 6

3 Design Rationale **6**

4 Design Constraints **7**

1 Introduction

1.1 Purpose

This document describes the architecture, components, and data design of the EswatiniHomes system. It translates the requirements defined in the Software Requirements Specification (SRS) into a technical blueprint that developers can use to implement the system.

1.2 Scope

This SDD covers the database schema, system architecture, PHP backend logic, and the interaction between the frontend UI and the MySQL database.

1.3 Design Stakeholders and Concerns

- **Developers:** Need detailed database schemas, file structures, and logic flows.
- **Project Manager:** Needs to ensure the design meets the SRS and is achievable within the timeline.
- **Maintainers:** Need a clear understanding of how the modules interact to fix future bugs.

2 Design Viewpoints

2.1 Context Viewpoint

The Context Viewpoint provides a "black box" perspective of the system, showing its boundaries and how it interacts with external entities (actors).

Actors:

- **Guest:** Browses properties, uses search filters.
- **Registered User:** Submits leads/contact forms.
- **Agent:** Uploads properties, tracks deals.
- **Admin:** Approves/rejects listings, manages users.

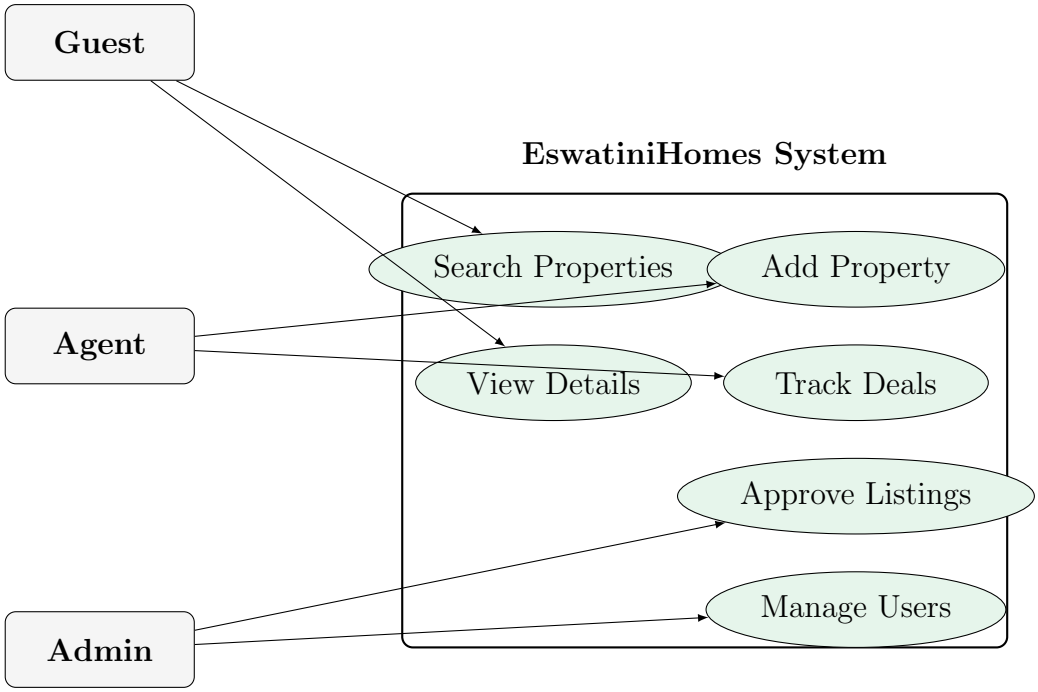


Figure 1: Use Case Diagram for EswatiniHomes

2.2 Composition Viewpoint

The system is composed of the following major modules. The directory structure is designed to separate concerns:

```
/property-listing
|-- /admin          # Admin dashboard and management scripts
|-- /assets         # Uploaded property images and CSS/JS files
|-- /auth           # Login, logout, and registration scripts
|-- /functions      # Helper scripts (e.g., add.php, view.php)
|-- /include        # Configuration and core logic (config.php)
|-- /pages          # Public pages (sale.php, rent.php,
   calculator.php)
|-- index.php       # Main public homepage
|-- delete.php      # Secure property deletion script
|-- database.sql    # Database schema and seed data
```

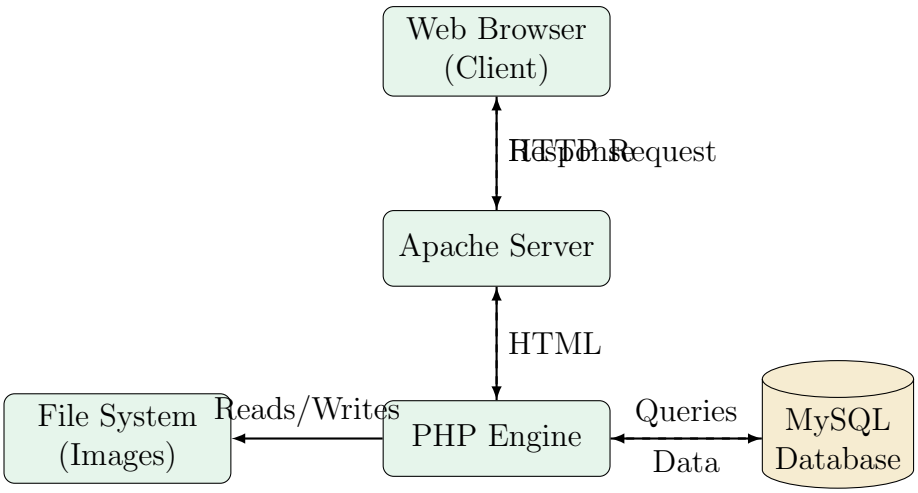


Figure 2: High-Level System Architecture

2.3 Logical Viewpoint (Data Design)

The database (`property_listing`) is the core of the application. Below is the logical design of the most critical tables.

2.3.1 Table: properties

Column	Type	Description
id	INT (PK)	Auto-incrementing unique identifier.
title	VARCHAR(150)	Name of the property.
price	DECIMAL(12,2)	Price in Eswatini Emalangeni (E).
property_type	ENUM	House, Apartment, Land, Commercial, Townhouse.
listing_type	ENUM	Sale or Rent.
status	ENUM	Pending, Approved, Rejected.
agent_id	INT (FK)	References `agents.id`.
image	VARCHAR(255)	Filename of the uploaded image.

Table 1: Logical Design of the `properties` Table

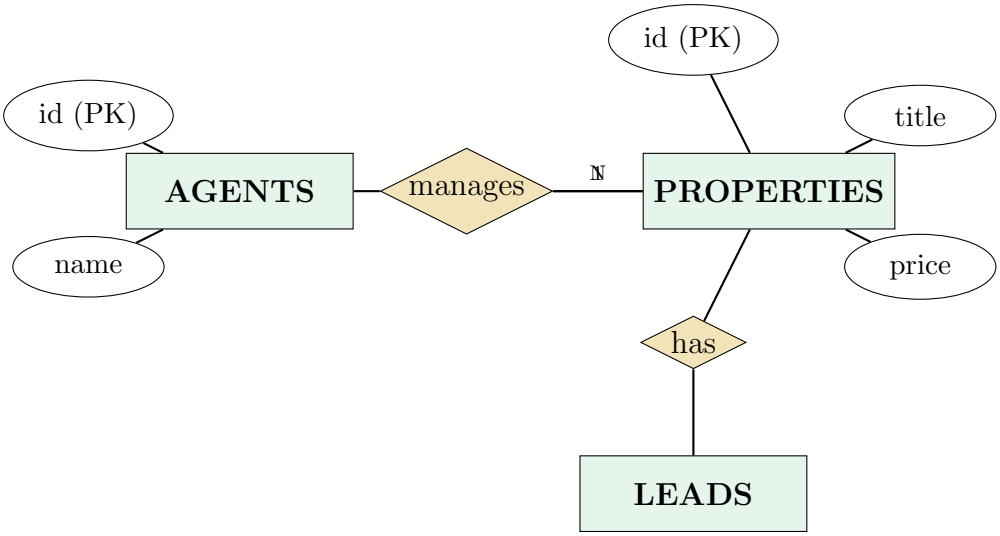


Figure 3: Simplified Entity-Relationship Diagram

2.4 Interface Viewpoint

This viewpoint describes how components interact with each other, specifically focusing on the Request/Response lifecycle.

2.4.1 Public Search Interface (`index.php`)

- **Input:** HTTP GET parameters (`q`, `type`, `listing`, `max_price`).
- **Processing:** The PHP script dynamically builds a SQL `WHERE` clause using prepared statements.
- **Output:** Renders an HTML grid of property cards. If no results, it outputs an "empty" state message.

2.4.2 Admin Delete Interface (`delete.php`)

- **Input:** HTTP POST request with `id` and `csrf_token`.
- **Constraints:** Must have an active admin session. Must pass CSRF validation.
- **Output:** Redirects to `admin/dashboard.php?updated=property`.

2.5 Interaction Viewpoint

This viewpoint describes the sequence of events for critical system operations.

2.5.1 Sequence: Property Deletion

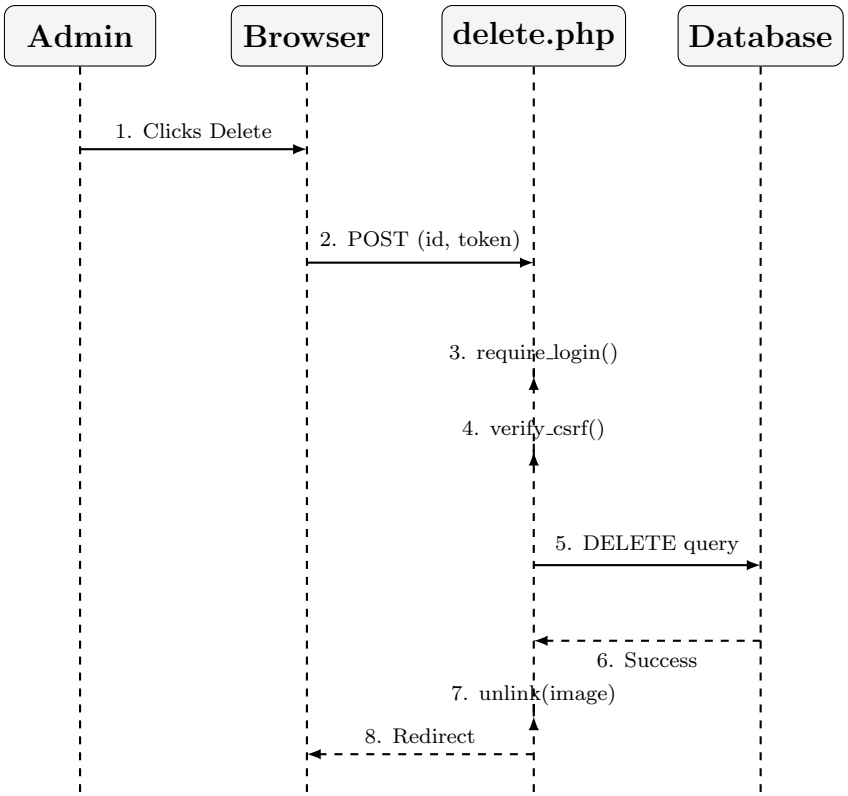


Figure 4: Sequence Diagram for Property Deletion

3 Design Rationale

This section explains the "Why" behind key design decisions.

- **Why PHP/MySQL (LAMP Stack)?**
  It is highly compatible with standard XAMPP environments, making local development and deployment cheap and easy for a project of this scale.

- **Why `mysqli` with Prepared Statements?**
  To prevent SQL injection attacks. The search bar on `index.php` takes user input, and prepared statements ensure that input is treated as data, not executable code.

- **Why separate `include/config.php`?**
  To ensure DRY (Don't Repeat Yourself) principles. Database credentials and helper functions (like `currency()` and `verify_csrf_token()`) are defined once and included everywhere, making the system easier to maintain.

- **Why delete the physical image file in `delete.php`?**
  To save server disk space. Deleting only the database row leaves orphaned image files on the server, which can accumulate and waste storage over time.

4 Design Constraints

- **Security Constraint:** The `xampp-control.ini` file requires administrator privileges to write to. The system must be run as an Administrator on Windows during local development.
- **File System Constraint:** Images must be stored in the `assets/` directory. The database only stores the filename (e.g., `house1.jpg`), not the full path, to allow for portability between local and live servers.
- **Database Constraint:** The `price` field uses `DECIMAL(12,2)` to ensure accurate financial calculations for commissions and property prices.